MemSnap: A Fast Adaptive Snapshot Algorithm for RMWable Shared-Memory

Prasad Jayanti
Dartmouth College
Hanover, NH, USA
prasad.jayanti@dartmouth.edu

Siddhartha Jayanti
Google Research
Cambridge, MA, USA
sjayanti@google.com

Sucharita Jayanti
Brown University
Providence, RI, USA
sucharita_jayanti@brown.edu

ABSTRACT

Shared-memory words in modern multiprocessors support read-modify-write (RMW) primitives, such as compare-and-swap, fetch-and-add, and fetch-and-store, in addition to standard reads and writes. Thus, checkpointing the shared-memory of a modern multicore requires a variant of the snapshot object, which needs to support only a single scanner, but which allows components to be updated via all the RMW operations supported by hardware.

In this paper, we design such an *RMWable*, single-scanner snapshot object, called MemSnap. MemSnap is time and space optimal: it requires only constant time per update, uses only memory linear in the number of components, and takes only linear time to scan all the components. Furthermore, MemSnap is *adaptive* [8, 29], meaning that it supports a *click* operation which quickly takes an implicit snapshot and returns nothing, and allows the scanner to efficiently read the values of the components in the snapshot through an *observe* method. Even from the point of view of adaptivity, the algorithm is space and time optimal, i.e., click takes only $O(1)$ time and each observation takes only $O(1)$ time. Consequently, MemSnap supports partial snapshots in time proportional to the number of components scanned. Additionally, MemSnap supports *parallel observation*—meaning that observations of the various components of the scan can be made in parallel; it also supports *dynamic addition* of components: meaning that new components can be added during the execution of an algorithm.

The MemSnap algorithm is efficient, brief, and elegant; however, its behavior is tantalizingly complex, since it exhibits future and far-future linearization. We prove its correctness via the recently introduced meta-configuration tracking technique [21].

CCS CONCEPTS

• **Computing methodologies** → **Concurrent algorithms**.

KEYWORDS

snapshot, checkpoint, backup, adaptive, parallel observation, meta-configuration tracking, RMWable, far future linearization

ACM Reference Format:
Prasad Jayanti, Siddhartha Jayanti, and Sucharita Jayanti. 2024. MemSnap: A Fast Adaptive Snapshot Algorithm for RMWable Shared-Memory . In *ACM Symposium on Principles of Distributed Computing (PODC '24), June 17–21, 2024, Nantes, France.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3662158.3662820

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
PODC '24, June 17–21, 2024, Nantes, France
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0668-4/24/06
https://doi.org/10.1145/3662158.3662820

1 INTRODUCTION

Checkpointing is a critical step in several systems and data processing pipelines. Prevalent examples include:

(1) **Memory Backups [24]:** The task is to capture an instantaneous image of either all or part of main memory. Capturing such an image is hard in the face of concurrent updates by tens or hundreds of processes with clocks running at the order of gigahertz.
(2) **Neural Network Training [10, 26]:** Large neural networks, including Large Language Models (LLMs), can consist of hundreds of billions of parameters and train over a period of days or weeks. Their training procedures consume a lot of time, power, and computational resources. Without checkpoints, the entire computation would need to be redone in case the pipeline crashes (due to a bug, power-fault, or underlying system crash). So, checkpointing the parameter weights at regular intervals during training is critical to saving researcher time, computational resources, and energy.
(3) **Debugging [25]:** Checkpoints of a running system can be useful in analyzing whether system invariants are maintained.

In essence, the checkpointing problem boils down to the snapshot problem [2], which calls for the design of an array of components, which can be atomically (i.e., linearizably) *scanned*. However, there are two caveats:

- **Read-modify-write (RMW) Primitives:** Along with read and write (i.e., load and store), modern hardware provides several read-modify-write (RMW) primitives, such as compare-and-swap (CAS), fetch-and-add (F&A), and fetch-and-store (FAS) [1]. These primitives are used widely in modern algorithms, yet traditional snapshot objects only allow modifications to components through write operations. Checkpointing requires an *RMWable* snapshot object, i.e., one whose individual components can be *updated* by any atomic primitive that hardware words support.
- **A single scanner suffices:** Traditional snapshot objects provide the ability for every process to scan. This capability to support concurrent scans is not needed for checkpointing. In particular, it suffices to have a single, designated, scanner process. We call such a snapshot algorithm a *single-scanner* algorithm.

In this paper, we design *MemSnap*, an RMWable, single-scanner snapshot object, where each component of the snapshot object

can be updated concurrently by an arbitrary number of processes. Our algorithm has optimal asymptotic space and time complexity: for a snapshot object with m components, the algorithm requires $O(m)$ words of memory, and performs updates in just $O(1)$ time (regardless of whether the update is a read, write, or an RMW operation such as CAS, F&A, or FAS), and performs a scan of all the components in just $O(m)$ time.

Adaptivity. Linear time, i.e., $\Omega(m)$, is the traditional lower bound for scanning m components, since even writing down the result of the scan takes m steps. Recently, Wei, Ben-David, Blelloch, Fatourou, Ruppert, and Sun [29], and concurrently, Bashari and Woelfel [8] introduced the notion of *adaptivity*, which allows for the breach of this lower bound. In their framework, the scan operation takes a snapshot *implicitly*, returning nothing. The scanner can later access the value of any component in the snapshot by calling the *observe* method on it. In this paper, we name this implicit scan a *click*, thereby deriving the *click and observe* adaptive snapshot interface. In order to compute a traditional scan, the scanner simply clicks once and subsequently observes all m components. Adaptivity can lead to efficiency, since a scanner that only needs to observe part of the snapshot only needs to pay for those observations. Furthermore, the scanner does not need to specify which components it will observe at scan-time. It can dynamically decide which components to observe, possibly depending on the values of previous observations.

In addition to being RMWable, our single-scanner snapshot object is also adaptive, with optimal time complexities for clicking and observation. In particular, our algorithm requires only $O(1)$ time for a click operation, and $O(1)$ time per observation.

Simplicity with a Challenge. MemSnap is elegant and efficient—displaying asymptotic time and space optimality with small constant factors. Thus, the algorithm is well suited for use in parallel algorithms and systems. Simultaneously, the linearization structure of the algorithm is tantalizingly complex. MemSnap exhibits *far-future linearizability* [27], meaning that the linearization points of certain operations cannot be determined even after the operations return. Thus, the algorithm is a great challenge problem for linearization proof techniques. We prove its correctness using *meta-configuration tracking*—a recently devised, invariant-based, complete, forward-reasoning proof technique due to Jayanti, Jayanti, Yavuz, and Hernandez [21]. We look forward to seeing MemSnap's simplicity, speed, and scalability contribute to practical applications.

1.1 Our Contribution

We give the formal specification of an RMWable, adaptive, single-scanner snapshot object, which we term a *Checkpointable Object*, and present a wait-free linearizable checkpointable object, which we call *MemSnap*. An m-component instantiation of our object has the following properties:

- Any process π can call the UPDATE$(i, uop, uarg)$ operation to update the ith component with the operation $uop(uarg)$, where uop can be a read, write, or any other RMW operation supported by hardware (e.g., CAS, FAS, F&A), and $uarg$ is uop's argument(s).
- A designated scanner process σ can call the CLICK() operation to implicitly scan all m components.
- The designated scanner process σ can call OBSERVE(i) to observe the value of the ith component of the latest scan.
- The time complexities of the operations—update, scan, and observe—are each $O(1)$.
- The space complexity of the object is $O(m)$.

We give an invariant-based proof of our algorithm via the meta-configuration tracking technique [21, 22].

Remark: Our algorithm exhibits two other features, which are of practical value to parallel algorithms. In any period of time where the scanner is not actively clicking, any number of processes can observe any of the components in parallel; we call this property *parallel observability*. Also, our algorithm allows new components to be added dynamically to a snapshot object at any time; we call this property *dynamic addition*.

2 RELATED WORK

The focus of this paper is on snapshotting RMWable memory with a single adaptive scanner. To our knowledge, there is no previous snapshot object that is both RMWable and adaptive. The closest related works are Jayanti's algorithm, which is RMWable but not adaptive [19], and Wei, Ben-David, Blelloch, Fatourou, Ruppert, and Sun's algorithm [29] and Bashari and Woelfel's algorithm [8], which are adaptive but not RMWable. We explain below how these algorithms compare with ours.

Jayanti's algorithm [19] is the only previous work on RMWable snapshot, but it is not adaptive. The algorithm supports multiple scanners and the scan operation runs in $O(m)$ time, which is optimal. The update operation however, is inefficient and takes $O(m)$ time. The space complexity is $O(m \log m)$. In comparison, our algorithm supports only a single scanner, but it is adaptive; all operations run in $O(1)$ time, and the space complexity is optimal at $O(m)$.

Wei et al. [29] and Bashari and Woelfel [8] introduced adaptive scans. Wei et al.'s adaptive algorithm supports multiple scanners and has the nice feature that any process can observe a component's value from any of the scans performed so far. Their algorithm however, supports only read and CAS operations for accessing/updating components, and not the other hardware operations (such as FAS and F&A). The click and update operations run in $O(1)$ time, but the time complexity of the observe operation and the space complexity are unbounded. In fact, the time complexity of observe and the space complexity remain unbounded even when their algorithm is tailored to the simpler case when there is only one scanner and the observe operation has to return only the value from the latest scan (for a fair comparison with our algorithm). Their work includes experimental results that show that space growth can be contained in practice if the algorithm is enhanced with an epoch based garbage collection scheme. In comparison, our snapshot object supports only a single scanner, but it is RMWable; all operations including observe run in $O(1)$ time, and the space complexity is optimal at $O(m)$, without any need for garbage collection or other complex mechanisms.

Bashari and Woelfel's adaptive algorithm supports multiple scanners, where each scanner's observe operation returns a value from its own latest scan. The algorithm however, implements only a single-writer snapshot, where no component can be written to

concurrently by more than one process. Because their snapshot object is single-writer, the issue of supporting RMW operations for updating components is moot. Their click operation runs in $O(1)$ time, but the observe and write operations take $O(\log m)$ time. The space grows unbounded in their first-cut algorithm, but the authors introduce additional pruning techniques to trade algorithmic complexity for a polynomial space bound of $O(m^3 \log m)$. In comparison, our snapshot object supports only a single scanner, but it is multi-writer and RMWable; all operations run in $O(1)$ time, and the space complexity is optimal at $O(m)$, without any need for complex space management schemes.

Generic Snapshot: Non-RMWable and Non-adaptive Algorithms. The snapshot object, first introduced by Afek, Attiya, Dolev, Gafni, Merritt, and Shavit [2] and Anderson [3], supports two operations: WRITE(i, v)—which writes the value v into the ith component, and SCAN()—which returns the values of all m components. In this formulation, individual components cannot be read without scanning the whole object, and individual components cannot be modified using synchronization primitives, such as CAS, FAS, and F&A. Snapshot algorithms abound in the literature, and they can be split across two-axes: (1) some algorithms are single-writer, meaning each component can only be updated by a single-process, while others are multi-writer; (2) some algorithms require hardware support for only reads and writes, while others use synchronization primitives like CAS, FAS, or F&A. There are several algorithms that fall into each of the four resultant categories: those that solve single-writer snapshot with reads and writes [2, 3, 5, 7, 12, 13, 18, 23], those that solve single-writer snapshot using stronger synchronization primitives [6, 9, 28], those that solve multi-writer snapshot with reads and writes [2, 4, 13, 16, 30], and those that solve multi-writer snapshot with stronger synchronization primitives [11, 19, 20].

Jayanti's algorithm was the first multi-scanner, multi-writer algorithm to achieve optimal time complexities for writing and scanning: the algorithm uses CAS to achieve $O(1)$-time updates and $O(m)$-time scans, with a space complexity of $O(mn^2)$, where n is the number of scanners [20]. Ba improved the space complexity to $O(mn)$ [11].

Attiya, Guerraoui, and Ruppert introduced the *partial snapshot* variant of the problem, in which the scanner can specify a subset of components $S \subseteq [m]$, and ask to only get a snapshot of those elements. Attiya et al.'s algorithm for partial snapshot requires $O(|S|^2)$ per scan operation, and $O(\overline{C}^2 \cdot |S|^2)$ time per write operation, where $\overline{C}$ is the interval contention in the given run. Notably, adaptive snapshot algorithms, such as ours can be used to efficiently take partial snapshots: simply click and observe the components in S.

3 MODEL

We work in the standard asynchronous shared-memory multiprocessor model. We use the read (a.k.a. load), write (a.k.a. store), and compare-and-swap (CAS) instructions in our algorithms. The CAS operation on a memory word X with arguments *old* and *new* is called as follows: CAS(X, old, new). The operation is atomic and has the following behavior: if $X = old$, then X's value is updated to *new* and *true* is returned to indicate that the operation successfully changed the value; otherwise (i.e., if $X \neq old$), the value of X is not changed and *false* is returned.

A couple of notes for practitioners: On modern x86 architectures, individual memory words are 64-bits, and so any hardware primitive can be applied on a standard 64-bit word. Usefully however, CAS can also be executed on 128-bit double-words, i.e., two adjacent words of memory [1, 17]. We will make use of this feature. (Note that this 128-bit CAS operation is *not* DCAS—a primitive that does CAS on two non-adjacent memory locations, which is not supported in modern architectures.) Our algorithm will use unbounded counters that count the number of CLICK operations performed. We ensure that each such counter can occupy a full 64-bit word; thus in practice, the data structure can support 2^{64} scans before overflow, i.e., even if a scan is performed every nanosecond (about once a clock-cycle), the counter will not overflow in 500 years.

Modern x86 architectures provide several other hardware primitives on individual memory words, such as:

- Fetch-and-store (FAS): the $r \leftarrow \text{FAS}(X, v)$ instruction captures X's value into the register r and changes X's value to v, all atomically.
- Fetch-and-add (F&A): the $r \leftarrow \text{F\&A}(X, n)$ instruction captures X's value into the register r and changes X's value to $X + n$, all atomically.

Our algorithm does not assume that hardware provides these primitives, but we do ensure that components of our checkpointable object have all the power that hardware memory-words have. In particular, our algorithm's UPDATE operation allows components to be updated by any primitive operation that hardware supports.

Our algorithm is *linearizable* [15], meaning that each operation appears to take place atomically at some point between its invocation and return. It is also *wait-free* [14], meaning that each operation terminates in a bounded number of its own steps.

We measure the efficiency of our algorithm by time and space complexity. The *time complexity* of an operation by a process π is the total number of steps executed by π between the invocation and return of that operation. The *space complexity* of an algorithm is the total number of memory words that the algorithm uses.

4 SPECIFICATION OF THE CHECKPOINTABLE OBJECT

We specify the interface of a *checkpointable object* with m components. The abstract state of this object is a pair (VAL, SNAP) of two sequences of length m. Informally, VAL$[j]$ is the value of the jth component and SNAP$[j]$ is the value of the jth component in the latest snapshot. Initially, VAL = SNAP = $\vec{v}$ for some initial sequence of values $\vec{v}$.

The checkpointable object supports three operations: UPDATE, CLICK, and OBSERVE. We specify the sequential behavior of each operation below, by describing the state (VAL′, SNAP′) that the object ends up in and the return value r if the operation is invoked in the state (VAL, SNAP).

(1) UPDATE$(i, uop, uarg)$ applies $uop(uarg)$ to the ith component, i.e.:
   - SNAP′ = SNAP
   - $\forall j \neq i$, VAL′$[j]$ = VAL$[j]$

- $(\text{VAL}'[i], r)$ are the new state and return value if the primitive operation $uop(uarg)$ were invoked on a word of memory whose value was $\text{VAL}[i]$.

(2) CLICK() refreshes the snapshot, i.e.: $\text{SNAP}' = \text{VAL}' = \text{VAL}$ and $\overline{r = \text{ack}}$.

(3) $\overline{\text{OBSERVE}(i)}$ returns the value of the ith component in the snapshot, i.e.: $(\text{VAL}', \text{SNAP}') = (\text{VAL}, \text{SNAP})$ and $r = \text{SNAP}[i]$.

In this paper, we are interested in a single-scanner implementation of this object for an arbitrary set of updater processes Π and a designated scanner σ; any process $\pi \in \Pi$ can perform an UPDATE, but only the designated scanner process σ is allowed to perform a CLICK or an OBSERVE.

To clarify the definition of this object, we give an example.

EXAMPLE 1. Consider a checkpointable object of three components, which initially have the values $[1, 2, 3]$. In the following table, we give a sequence of operations and describe what the new state and response would be if these operations are executed sequentially, from top to bottom.

Operation	New State	Response
Initially	$([1, 2, 3], [1, 2, 3])$	
UPDATE(2, CAS, (2, 20))	$([1, 20, 3], [1, 2, 3])$	*true*
CLICK()	$([1, 20, 3], [1, 20, 3])$	ack
UPDATE(3, FAS, 30)	$([1, 20, 30], [1, 20, 3])$	3
UPDATE(1, F&A, 10)	$([11, 20, 30], [1, 20, 3])$	1
OBSERVE(3)	$([11, 20, 30], [1, 20, 3])$	3
UPDATE(1, $read$, $\perp$)	$([11, 20, 30], [1, 20, 3])$	11
UPDATE(1, $write$, 10)	$([10, 20, 30], [1, 20, 3])$	ack
OBSERVE(2)	$([10, 20, 30], [1, 20, 3])$	20

5 ALGORITHM

We present the code of MemSnap, our checkpointable object as Algorithm 1. The algorithm uses just three variables: a *principle array* A of m memory words, a *secondary array* B of m (val, seq)-pairs, and a *scan counter* X, which is an integer. The beauty of the algorithm is that it uses just these two arrays of length m and a counter to represent a checkpointable object, whose abstract representation itself requires two arrays of length m, namely, VAL and SNAP.

Intuitively, A corresponds to VAL, B corresponds to SNAP, and X counts the number of snapshots that have been taken. We note that these correspondences are not exact. As we will see in the proof, the true relationship between the arrays A and B of the implementation and the abstract state (VAL, SNAP) is subtle and complex, but thinking of the variables with these correspondences will aid intuition as you read the algorithmic description.

5.1 Description of the Algorithm

To initialize a checkpointable object of m components to initial values $\vec{v} = (v_1, v_2, \ldots, v_m)$, we set $X = 0$, $A[i] = v_i$ and $B[i] = (v_i, 0)$ for each $i \in [m]$.

The main idea behind the algorithm is that the words of the array A directly represent the m components. In particular, if an UPDATE wants to update component i with the primitive operation $uop(arg)$—e.g., execute UPDATE$(i, \text{FAS}, 5)$—it will simply execute the primitive on $A[i]$—i.e., $A[i].\text{FAS}(5)$—and return the resultant response (Lines 7 and 8). We call the point at which UPDATE performs the primitive at Line 7, the time of *explicit execution* of UPDATE. (We will explain the FORWARD(i) method later.)

Since CLICKS must be fast, the scanner simply increments the scan counter X to $X + 1$, and returns (Lines 1 and 2). Intuitively, we think of the associated snapshot as consisting of the values in A at time of the scan counter increment. Ideally, the m values in the principle array A would be instantaneously transferred to the secondary array B array at the time of the scan counter increment; of course, this does not happen in reality, but for now imagine it does. That is, imagine that each $B[i]$ instantaneously becomes $(A[i], X + 1)$, where $X + 1$ is the new incremented value of the counter.

In the above mentioned ideal scenario, an observation of the ith component—i.e., OBSERVE(i) operation—would simply return $B[i].val$ (Lines 4 and 5).

Of course, the catch in the above description is that the values of the principal array A do not, in reality, instantaneously transfer to the secondary array B. So, what do we do to address this catch? Our idea is that both updaters and observers can account for this catch. In particular, when a process π starts an update to location i, it first checks whether the scan counter has incremented past $B[i].seq$. If so, then π knows that the current value in $A[i]$ should be part of the scan that incremented the scan counter, so it transfers the value in $A[i]$ to $B[i].val$ along with setting $B[i].seq$ to the new scan count, before the explicit execution. This whole process of checking and transferring is done through the helper method FORWARD(i) (Line 6). Similarly, when the scanner σ starts an observation of location i, it knows that the value corresponding to the latest snapshot might not yet have been transferred to $B[i]$, so it also performs a FORWARD(i) before returning the value in $B[i].val$ (Line 3).

What remains is the explanation of the FORWARD method. As described before, a process π executes FORWARD(i) with the intent of checking whether the value in $B[i]$ corresponds to the latest snapshot, i.e., it wants to update $B[i]$ to $(A[i], X)$ if $B[i].seq < X$. Thus, in forwarding, π reads $B[i]$ and reads X to determine whether the current snapshot value is *outdated* (Lines 9 and 10). If it is outdated, it reads $A[i]$ (Line 11), and attempts to transfer the updated snapshot into $B[i]$ (Line 12). While there is only a single scanner, many updating processes may be trying to help the scanner update $B[i]$, so synchronizing via a CAS is pivotal at Line 12. If the attempted CAS on Line 12 fails, that means some other process must have successfully updated the snapshot in $B[i]$. So, can process π simply return after a failed CAS? No! Observe that $B[i]$ can, in general, get very outdated. For example, $B[i]$ can still be in its initial state with $B[i].seq = 0$ even after several scans have occurred and thus $X = 25$. In particular, this can be the case if location i has not been updated or observed, or even if there are many updaters working on location i, but they are simply running very slowly. For example, some such slow updater π_{old} could be poised at Line 12, ready to CAS, having read a smaller value of X such as 6, while another recent updater π_{recent} has come to the CAS at Line 12 having read the latest value of $X = 25$. If π_{old} performs its CAS first, then π_{new}'s CAS will fail; however, while π_{old} has updated $B[i]$, the value it has updated $B[i]$ to is still outdated. Thus, π_{new} must try to update $B[i]$ for a second time (for-loop before Line 9). We show in the proof, that trying twice is sufficient.

Algorithm 1 *MemSnap* Algorithm: implementation of a checkpointable object of m components.

```
    class Checkpointable
        instance variable Array A                          ▷ array of m memory words
        instance variable Array B                          ▷ array of m (val, seq) pairs
        instance variable int X
        Initially:
            X = 0
            For all i ∈ [1, m]:
                A[i] = B[i].val = v_i, where (v⃗, v⃗) is AdaptiveSnapshot's initial state
                B[i].seq = 0
        procedure CLICK()
1:          X ← X + 1                                      ▷ Since only the scanner writes to X, this step can be considered atomic.
2:          return ack
        procedure OBSERVE(int i)
3:          FORWARD(i)
4:          r ← B[i].val
5:          return r
        procedure UPDATE(int i, operation uop, argument uarg)
6:          FORWARD(i)
7:          r ← A[i].uop(uarg)
8:          return r
        procedure FORWARD(int i)
            for k ← 1 to 2:
9:              b ← B[i]
10:             x ← X; if b.seq = x then return
11:             a ← A[i]
12:             if CAS(B[i], b, (a, x)) then return
            return
```

5.2 Key Algorithmic Insights Before the Proof

Before describing the formal invariants of the algorithm and its proof, we informally observe a series of crucial features of the algorithm to help understand how the CLICK, OBSERVE, and UPDATE operations should be linearized.

- The scan counter X never lags behind the sequence numbers in the secondary array: for all $i \in [m]$, $B[i].seq \leq X$. When the scanner increments X at Line 1, X exceeds $B[i].seq$. When a OBSERVE or UPDATE operation on component i executes the FORWARD method (at Line 3 or Line 6), the forward method attempts to equalize X and $B[i].seq$.
- Since the scanner σ is the only process to execute the CLICK and OBSERVE operations, the CLICK and OBSERVE operations are never concurrent. Therefore, X remains unchanged while a OBSERVE operation (on any component i) executes the FORWARD method. Consequently, when the FORWARD method, executed from Line 3, completes, it indeed succeeds in helping $B[i].seq$ catch up with X. In other words, at Line 4, the condition $B[i].seq = X$ holds.
- The story is different when FORWARD is executed at Line 6 by an UPDATE operation. In particular, since X can be changed by CLICK operations concurrently with an UPDATE, the condition $B[i].seq = X$ does not necessarily hold when the FORWARD at Line 6 completes. What then does the FORWARD at Line 6 accomplish? To answer this question, let $\hat{x}$ be the value of X at the time the FORWARD method on component i is called at Line 6. Then, when the forward method completes, it ensures that $B[i].seq$ catches up to at least $\hat{x}$, i.e., the condition $B[i].seq \geq \hat{x}$ holds when the forward method completes.
- The next observation is about the correspondence between the principal and secondary arrays A and B and the abstract state (VAL, SNAP). Intuitively, we would want that $A[i] = \text{VAL}[i]$ and $B[i].val = \text{SNAP}[i]$. Such a correspondence does not always hold, but it does hold when $B[i].seq = X$. That is, if $B[i].seq = X$ it is not only true that $B[i].val = \text{SNAP}[i]$ but it is also true that $A[i] = \text{VAL}[i]$. On the other hand, When $B[i].seq \neq X$ (i.e., $B[i].seq < X$), it is not only true that $B[i].val$ may differ from $\text{SNAP}[i]$, but it is also true that $A[i]$ may not contain the value of $\text{VAL}[i]$. This fact implies that *updaters do not always linearize at the time of their explicit action* (i.e., at Line 7). How then do UPDATE operations linearize? We will address this intriguing question below, after first explaining how to linearize CLICK and OBSERVE, which are simpler to understand.
- The CLICK operation by the scanner, which takes a fresh snapshot, is linearized at Line 1, i.e., at the instant that X is incremented. Intuitively, the increment of X causes the condition $B[i].seq < X$ to hold, for every component i, which implies that $B[i].val \neq \text{SNAP}[i]$, i.e., $B[i].val$ does not reflect component i's value in the latest snapshot. The task of depositing the snapshot's value (at component i) in $B[i].val$, while simultaneously equalizing $B[i].seq$ and X, becomes the responsibility of subsequent OBSERVE and UPDATE operations on component i.
- The OBSERVE(i) operation, which is required to return the ith component's value in the latest snapshot, is linearized at Line 4. Informally, this linearization is justified because, as we already explained, the condition $B[i].seq = X$ holds at Line 4, which means that $B[i].val = \text{SNAP}[i]$.
- Finally, we turn to the linearization of UPDATE operations, which is very intricate. Below we provide the intuition by presenting a scenario. Throughout the scenario, we fix our focus on an arbitrary component $i \in [m]$.

Suppose that the scanner clicks a fresh snapshot at some time τ, i.e., its CLICK operation increments X at Line 1 at time τ. At this point, we have $B[i].seq < X$, so $B[i].val$ does not reflect component i's value in the latest snapshot. Let $v^{\#}$ denote the value in $A[i]$ at time τ, and let S denote the set of pending UPDATE operations on component i at time τ; these are UPDATEs on component i that started before τ but have not executed their explicit action (Line 7) by τ.

As the FORWARD method is executed (on component i by OBSERVE or UPDATE operations), some execution of the forward method can cause $B[i].seq$ to catch up with X. Let $t > \tau$ be the earliest instant when $B[i].seq$ becomes equal to X, and let v^* be the value in $B[i].val$ at t. This means that some forward method by some process π must have read v^* from $A[i]$ at Line 11 at some time t', which is between τ and t, and planted that value in $B[i].val$, while simultaneously setting $B[i].seq$ to X, via a successful CAS at Line 12. At t, since $B[i].seq = X$, the value v^* in $B[i].val$ becomes component i's value in the latest snapshot, taken at τ.

A key observation is that the value in $A[i]$ changed from $v^{\#}$ at time τ to v^* at time t' because of the explicit actions (i.e., RMW operations on $A[i]$ at Line 7) performed during the interval (τ, t') by some subset $S' \subseteq S$ of processes in a certain order, which is some permutation α of S'. To account for the fact that the UPDATE operations in S' are reflected in the value of the snapshot taken at τ, this set S' of UPDATE operations are linearized just before the instant τ, in the order given by α. Since the operations in S' are pending at τ, their linearization point (of just before τ) falls within their interval of execution, so the linearization is legitimate.

5.3 Challenges for the Proof

We now explain why our disarmingly simple looking algorithm is challenging to prove correct.

An explosion of possibilities. As just discussed in the last bullet of the previous subsection, if S is the set of pending updates at a time τ that a snapshot is clicked, any permutation α of any subset S' of S can end up linearizing just before τ. Since our algorithm works for an arbitrary set of processes, the size of S' can be any (unbounded) integer. Thus, when a snapshot is taken at time τ, the snapshot's value will be determined only in the future, and that value can turn out to be the result of applying any permutation of any subset of an unbounded set of operations to the primary array's values at τ. This explosive number of possibilities for the linearization makes the proof challenging.

Far-future linearization. It is well understood in the verification community that algorithms are hard to prove correct if they exhibit *future linearization*, i.e., if it is not always possible to identify the linearization point of an operation as it occurs. Yet harder to prove, are algorithms that exhibit *far-future linearization*, for which it is not possible to identify the linearization point of some operation even after the operation has completed [27]. Our algorithm exhibits far-future linearization, as demonstrated by the following scenario.

- Initially, $X = 0$, $A[1] = 5$, $B[1].val = 5$, $B[1].seq = 0$
- A process π calls UPDATE$(1, \text{write}, 10)$, completes Line 6 and is poised at Line 7.
- The scanner σ executes CLICK, thereby incrementing X to 1. Notice that π's UPDATE is pending at the time of the scanner's execution of CLICK.
- A process π_1 calls UPDATE$(1, *, *)$. In the UPDATE method, it calls FORWARD from Line 6, executes Lines 9, 10, and 11 of FORWARD, and stops just before executing Line 12. Notice that if π_1 were allowed to execute Line 12, its CAS on $B[1]$ would succeed, setting $B[1].seq$ to 1 and $B[1].val$ to 5.
- Process π, poised at Line 7, proceeds to execute Lines 7 and 8, returning *ack* and completing its UPDATE operation.
- A process π_2 calls UPDATE$(1, *, *)$. In the UPDATE method, it calls FORWARD at Line 6, executes Lines 9, 10, and 11 of FORWARD, and stops just before executing Line 12. Notice that if π_2 were allowed to execute Line 12, its CAS on $B[1]$ would succeed, setting $B[1].seq$ to 1 and $B[1].val$ to 10.

By the end of this scenario, π's UPDATE$(1, \text{write}, 10)$ operation has completed, but its linearization point cannot be determined yet; in particular, as we explain below, its linearization depends on which of π_1 and π_2 will execute Line 12 first.

Suppose that π_1 executes Line 12 first. Then, its CAS will succeed, setting $B[1].seq$ to 1 and $B[1].val$ to 5. Suppose that a process π_3 calls OBSERVE(1) and executes it to completion. It is easy to verify that π_3's OBSERVE(1) returns 5. Since π's update of 10 is not reflected in π_3's observation of the snapshot, π's UPDATE must linearize *after* the scanner's CLICK operation.

Suppose instead that π_2 executes Line 12 first. Then, its CAS will succeed, setting $B[1].seq$ to 1 and $B[1].val$ to 10. Suppose that a process π_3 calls OBSERVE(1) and executes it to completion. It is easy to verify that π_3's OBSERVE(1) returns 10. Since π's update of 10 is reflected in π_3's observation of the snapshot, π's UPDATE must linearize *before* the scanner's CLICK operation.

We conclude that the algorithm exhibits far-future linearization.

We believe that the algorithm's brevity and elegance, together with its behavioral complexity because of far-future linearization and explosive number of possibilities for linearization, makes it an ideal case study to demonstrate and compare the efficacy of various proof techniques.

6 PROOF

We prove our algorithm's linearizability using the *Meta-Configuration Tracking* method, which is a universal and complete technique for proving linearizability, recently devised by Jayanti, Jayanti, Yavuz, and Hernandez [21, 22].

In accordance with the meta-configuration tracking technique, we augment our algorithm with a set-valued *meta-configuration variable* P, which—as we will later demonstrate—tracks a set of possible linearizations of the current run. The augmentation is presented as Algorithm 2. Along with P, our augmentation also introduces an additional ghost variable (a.k.a. history variable) $\widehat{x}$, which makes it easier to formalize invariants. Importantly, it is trivial to check that P and $\widehat{x}$ do not affect the behavior of the original program variables, thus Algorithm 2's behavior is identical to that of our original algorithm, Algorithm 1. Now, we briefly summarize

the idea behind the meta-configuration tracking proof method, and the main theorem proved in that paper (which is rather intuitive).

We call the elements of the meta-configuration tracking variable P *possibilities*. Each possibility $p \in P$ has the fields: $p.\text{VAL}$, $p.\text{SNAP}$, and $p.\pi$ for each $\pi \in \Pi \cup \{\sigma\}$. The augmentation is designed to ensure that, at every point of time t, every possibility p in the set P corresponds to a linearization L_p of the run up to time t, in the sense that:

(1) $(p.\text{VAL}, p.\text{SNAP})$ is the abstract state of the object at time t corresponding to the linearization L_p.
(2) For each process $\pi \in \Pi \cup \{\sigma\}$:
   - $p.\pi = \perp$ if p is currently executing no operation or is executing an operation that, according to L_p, has not yet linearized.
   - $p.\pi \neq \perp$ if π is executing an operation that, according to L_p, has linearized with response $p.\pi$.

Given that the augmentation is properly defined such that the possibilities in P satisfy this correspondence, the main theorem in the meta-configuration tracking paper observes that: since each possibility $p \in P$ corresponds to a linearization, the original algorithm (Algorithm 1) is linearizable if the statement $P \neq \emptyset$ is an invariant of the augmented algorithm (Algorithm 2). Indeed, we will demonstrate the linearizability of our checkpointable object by showing that $P \neq \emptyset$ is an invariant of Algorithm 2.

How do we maintain P such that its elements correspond to linearizations in the above mentioned way? We can easily do this by following two rules put forth in the Jayanti et al. paper. To describe the rules, consider the execution of a line ℓ by a process π, and let P and P' be the values of the augmented meta-configuration tracking variable immediately preceding and succeeding the execution of ℓ. We ensure the following two properties for each line ℓ of the augmented algorithm.

- If ℓ is an internal line of code, we ensure that every new possibility $p' \in P'$ is a result of linearizing some (possibly, none) unlinearized operations in an old possibility $p \in P$.
- If ℓ is a return statement that returns a value r as the response to one of the interface operations—i.e., UPDATE, CLICK, or OBSERVE (but not FORWARD), then we filter out all possibilities p that do not recognize π as linearized with $p.\pi = r$. That is, we ensure that $P' = \mathit{Filter}(P, \pi, r)$, where the filter function is defined as:

$$\mathit{Filter}(P, \pi, r) = \{p' \mid \exists p \in P : \ p.\pi = r, \ p' \text{ is } p \text{ except } p'.\pi = \perp\}$$

6.1 Description of the Augmented Algorithm

We now describe our augmentation, presented as Algorithm 2. Notice that the code is identical to that of Algorithm 1, excepting two changes: (1) the red augmenting lines defining the behavior of P and $\widehat{x}$ have been added; (2) the code has been explicitly marked as that of process π, and all local variables have been given the subscript π to make it easier for the formal invariants to differentiate between the local variables of different processes.

In the augmentation, each process $\pi \in \Pi \cup \{\sigma\}$ has a local variable $\widehat{x}_\pi$. This variable is used in two augmenting statements of the form $\widehat{x}_\pi \leftarrow X$, which occur along with the invocations of OBSERVE and UPDATE. These simply record the value of the snapshot counter X at the time of the invocations in order to make it easier to state invariants.

The more interesting auxiliary variable that the augmentation introduces is the set of possibilities P. In the initial configuration, no process has a pending operation, and no operations have taken place. So, there is a single trivial linearization, and thus P has a single possibility p_0, which corresponds to this linearization. In particular $(p_0.\text{VAL}, p_0.\text{SNAP})$ is the initial state of the object, and $p_0.\mu = \perp$ for all processes $\mu \in \Pi \cup \{\sigma\}$, since all processes are initially idle.

The augmentations occurring along with return statements (Lines 2, 5, and 8) are also easily explained. They simply filter out possibilities that are inconsistent with the return values as described in the rules above.

This leaves the interesting part of the augmentation—Lines 1, 4, and 7, which correspond to linearization points.

- Line 1 is the most interesting line from the point of view of linearizations. As we discussed earlier, the scanner σ's CLICK linearizes at Line 1 and, in addition, an arbitrary subset S of unlinearized updaters may also linearize at this point in an arbitrary order α, right before the scanner σ linearizes (notice that $\pi = \sigma$ for the CLICK and OBSERVE methods). Thus, the set of possibilities P' right after Line 1 executes corresponds to all possibilities p' that result from taking any possibility $p \in P$, identifying any subset S of unlinearized updaters in p, linearizing them in any order α, and then linearizing the scanner σ. The formal statement of this step uses the following definitions, where P^* is the universe from which possibilities are drawn:

  Definition 6.1. $\mathit{UnLin}(p \in P^*)$ is the set of active updaters that have not yet linearized in possibility p. Formally: $\mathit{UnLin}(p \in P^*) \triangleq \{\pi \in \Pi \mid pc_\pi \in [6, 8] \ \wedge \ p.\pi = \perp\}$

  Definition 6.2. If p and p' are possibilities and α is a permutation of a set of active processes that have not yet linearized in possibility p, then $\mathit{TransitionsOK}(p, \alpha, p')$ is a predicate that states that possibility p' results if, starting from possibility p, processes in $\alpha = \pi_1, \pi_2, \ldots, \pi_{|\alpha|}$ apply their respective update operations $op_1, op_2, \ldots, op_{|\alpha|}$, in that order.

  We also use the notation $\alpha \cdot \sigma$ to denote the sequence obtained by appending σ to the end of the sequence α.

- The augmentation at Line 4 simply states that every possibility $p \in P$ is updated to reflect that the OBSERVE operation linearizes (by itself) at this line.

- As we discussed in the augmentation to Line 1, some updates linearize right before a click. The augmentation at Line 7 expresses that all other updaters linearize at their point of explicit action. In particular, it states that every possibility $p \in P$, where the updater π which is currently executing its explicit action is unlinearized, is updated to reflect that π linearizes at this point.

Since the augmentation in Algorithm 2 indeed satisfies the two rules laid out in the Jayanti et al. paper, we are left to prove that $P \neq \emptyset$ is an invariant of this augmented algorithm to finish the proof of linearizability of Algorithm 1.

Algorithm 2 Meta-Configuration Tracking Augmentation of Algorithm 1, which is an implementation of a checkpointable object of m components. Code shown is for a process π; in the code for the CLICK and OBSERVE methods, π must equal σ, the designated scanner process.

```
    class AdaptiveSnapshot
        instance variable  Array  A                                   ▷ array of m values
        instance variable  Array  B                                   ▷ array of m (val, seq) pairs
        instance variable  int    X
        Initially:
            X = 0
            For all i ∈ [1, m]:
                A[i] = B[i].val = v_i, where (v⃗, v⃗) is AdaptiveSnapshot's initial state
                B[i].seq = 0
            P = {p_0}, where (p_0.VAL, p_0.SNAP) = (v⃗, v⃗) and ∀μ ∈ Π ∪ {σ} : p_0.μ = ⊥
        procedure CLICK()
1:          X ← X + 1
            P ← {p' | ∃p ∈ P : ∃S ⊆ UnLin(p) : ∃α ∈ Perm(S) : TransitionsOK(p, α · π, p')}
2:          return ack
            P ← Filter(P, π, ack)
        procedure OBSERVE(int i_π)
            x̂_π ← X
3:          FORWARD(i_π)
4:          r_π ← B[i_π].val
            P ← {p' | ∃p ∈ P : (p.π = ⊥) ∧ TransitionsOK(p, π, p')}
5:          return r_π
            P ← Filter(P, π, r_π)
        procedure UPDATE(int i_π, operation uop, argument uarg)
            x̂_π ← X
6:          FORWARD(i_π)
7:          r_π ← A[i_π].uop(uarg)
            P ← P ∪ {p' | ∃p ∈ P : (p.π = ⊥) ∧ TransitionsOK(p, π, p')}
8:          return r_π
            P ← Filter(P, π, r_π)
        procedure FORWARD(int i_π)
            for k_π ← 1 to 2:
9:              b_π ← B[i_π]
10:             x_π ← X; if b_π.seq = x_π then return
11:             a_π ← A[i_π]
12:             if CAS(B[i_π], b_π, (a_π, x_π)) then return
            return
```

7 INVARIANT AND ITS PROOF

In this section, we state and prove an invariant of Algorithm 2 that strengthens (and thereby implies) the $P \neq \emptyset$ invariant that implies the linearizability of our checkpointable object. We are able to decompose the invariant into two parts and prove the first part without any dependence on the second part, which simplifies the proofs. The first part, which we call *Forwarding Invariant* $\mathcal{I}_1$, captures the properties of the FORWARD method. We call the second part *Possibilities Invariant* $\mathcal{I}_2$, and it identifies the set of possibilities that ensures the algorithm's linearizability.

7.1 *Forwarding* Invariant $\mathcal{I}_1$

The Forwarding Invariant $\mathcal{I}_1$ consists of nine parts (sub-statements) and is stated in Figure 1 using the following notation:

- $[a, b]$, for any integers a and $b \geq a$, is the set $\{a, a+1, \ldots, b\}$.
- pc_π is process π's program counter, i.e., the line that π executes next.
- When π is at Line 10 of the FORWARD method, $pc_\pi = 3.10$ if FORWARD was called from Line 3, and $pc_\pi = 6.10$ if the call was from Line 6. However, since only the scanner calls from Line 3 (and updaters from Line 6), we use the simpler notation of $pc_\pi = 10$.
- $pc_\pi = 3$ (resp. $pc_\pi = 6$) is a shorthand for $pc_\pi \in \{3.9, 3.10, 3.11, 3.12\}$ (resp. $pc_\pi \in \{6.9, 6.10, 6.11, 6.12\}$).

We now describe the invariant informally.

Part (1) states that, for any j, $B[j].seq$ never exceeds X.

Part (2) states that the scanner's history variable $\widehat{x}_\sigma$ remains equal to X throughout the length of an OBSERVE operation.

Part (3) states that the history variable $\widehat{x}_\mu$ of a process μ executing an UPDATE operation is at most X during that UPDATE.

Part (4) states that when a process μ, which may be an updater or the scanner, is at lines 11 or 12 of the FORWARD method, the relationship $X > x_\mu > b_\mu.seq$ holds.

Part (5) states that, while a process μ is in one of the two for-loop iterations of the FORWARD method, if $B[i_\mu]$ changes after μ reads it into b_μ at Line 9, then the sequence number currently in $B[i_\mu]$ must be strictly greater than what μ found at Line 9.

Parts (6) and (7) capture the workings of the FORWARD method whose mission, when called by a process μ on component i_μ, is to ensure that $B[i_\mu].seq \geq \widehat{x}_\mu$ holds when the forward method returns.

Part (6) states that, if a process μ is in the first iteration of the for-loop of the FORWARD method with $B[i_\mu]$ having changed after μ read it into b_μ at Line 9 or if μ is at Line 9 in the second iteration, then either the mission is already accomplished (i.e., $B[i_\mu].seq \geq \widehat{x}_\mu$ holds) or it is about to be accomplished, i.e., every process μ' that is poised to change $B[i_\mu]$ is guaranteed to set $B[i_\mu].seq$ to be at least as big as $\widehat{x}_\mu$. (We consider μ' to be poised to change $B[i_\mu]$ if it is at

(1) $\forall j \in [1,m] : X \geq B[j].seq$
(2) $pc_\sigma \in [3,5] \Rightarrow X = \widehat{x}_\sigma$
(3) $\forall \mu \in \Pi : pc_\mu \in [6,8] \Rightarrow X \geq \widehat{x}_\mu$
(4) $\forall \mu \in \Pi \cup \{\sigma\} : pc_\mu \in [11,12] \Rightarrow X \geq x_\mu > b_\mu.seq$
(5) $\forall \mu \in \Pi \cup \{\sigma\} : (pc_\mu \in [10,12] \wedge B[i_\mu] \neq b_\mu) \Rightarrow (B[i_\mu].seq > b_\mu.seq)$
(6) $\forall \mu \in \Pi \cup \{\sigma\} : ((((pc_\mu \in [10,12] \wedge B[i_\mu] \neq b_\mu \wedge k_\mu = 1) \vee (pc_\mu = 9 \wedge k_\mu = 2)) \Rightarrow$
$((B[i_\mu].seq \geq \widehat{x}_\mu) \vee$
$\forall \mu' \in \Pi \cup \{\sigma\} : ((i_\mu = i_{\mu'} \wedge pc_{\mu'} \in [11,12] \wedge B[i_\mu] = b_{\mu'}) \Rightarrow x_{\mu'} \geq \widehat{x}_\mu)))$
(7) $\forall \mu \in \Pi \cup \{\sigma\} : (pc_\mu \in [10,12] \wedge B[i_\mu] \neq b_\mu \wedge k_\mu = 2) \Rightarrow (B[i_\mu].seq \geq \widehat{x}_\mu)$
(8) $pc_\sigma = 4 \Rightarrow B[i_\sigma].seq = X$
(9) $\forall \mu \in \Pi : pc_\mu = 7 \Rightarrow B[i_\mu].seq \geq \widehat{x}_\mu$

Figure 1: Statement $\mathcal{I}_1$: the *Forwarding Invariant* that captures the workings of the FORWARD method

Line 11 or 12 and if it were to proceed to execute Line 12, the CAS at that line would succeed and change $B[i_\mu]$.)

Part (7) states that if a process μ is already in the second iteration of the for-loop (of the FORWARD method) and its CAS is doomed to fail because $B[i_\mu] \neq b_\mu$, then the mission is already accomplished, i.e., $B[i_\mu].seq \geq \widehat{x}_\mu$.

Part (8) states: by the time the scanner returns to Line 4 from FORWARD, the sequence number in $B[i_\sigma].seq$ catches up to X.

Part (9) states that by the time an updater process μ returns from FORWARD to Line 7, $B[i_\mu].seq \geq \widehat{x}_\mu$ holds.

LEMMA 7.1. *Statement $\mathcal{I}_1$ in Figure 1 is an invariant of Algorithm 2.*

PROOF. The proof by induction is omitted for lack of space. □

7.2 *Possibilities* Invariant $\mathcal{I}_2$

The Possibilities Invariant $\mathcal{I}_2$, which helps argue that the algorithm is linearizable, is stated in Figure 2. It defines a certain set Q of possibilities and asserts that this set Q is a nonempty subset of the set P of possibilities tracked by Algorithm 2. This assertion implies that P is nonempty, which in turn implies (by the tracker theorem of [21]) that Algorithm 1 is linearizable. We now explain the motivation for how Q is defined.

The set Q is built from a "base possibility" p^* where, as defined in the figure, the abstract state (VAL, SNAP) is $(A, B.val)$, and each process π's ongoing operation is linearized only if π is at the return statement, i.e., π is at line 2, 5, or 8.

We know that, for any component j, either $B[j].seq < X$ or $B[j].seq = X$. As explained earlier, if $B[j].seq < X$, any subset of ongoing update operations on component j that started *before* the latest CLICK may linearize in any permutation α, immediately before the instant when the latest CLICK took place. Thus, if $B[j].seq < X$, VAL$[j]$ and SNAP$[j]$ are both obtained by applying the update operations of the processes in α to $A[j]$. On the other hand, if $B[j].seq = X$, VAL$[j]$ and SNAP$[j]$ are simply $A[j]$ and $B[j].val$, respectively.

To capture the above insight, Q's definition considers $UnLin(p^*)$, which is the set of processes that have ongoing update operations in p^*, and intersects it with the set $\{\pi \in \Pi \mid \widehat{x}_\pi < X\}$, to obtain the set of processes that have ongoing update operations *that started before the latest CLICK*; it then takes an arbitrary subset S of this set and an arbitrary permutation α of S. Intuitively, α is the sequence of processes that linearize just before the latest CLICK. To account for the fact that operations on components j for which $B[j].seq = X$ should not appear in S, the definition stipulates that S not include update operations on such components by introducing the clause $\forall \pi \in S : i_\pi \neq j$.

A final insight is that if a process μ is about to perform a successful CAS at Line 12 that will cause $B[j].seq$ to become equal to X, the value a_μ that it is about to install in $B[j].val$ has the potential to be component j's value in the latest snapshot (i.e., $a_\mu =$ SNAP$[j]$) and $A[j]$ has the potential to be VAL$[j]$. In this case too, to account for the fact that no ongoing updates on component j need to be linearized before the latest CLICK, we include the clause $\forall \pi \in S : i_\pi \neq j$.

The definition of Q is based on the above insights.

LEMMA 7.2. *Statement $\mathcal{I}_2$ in Figure 2 is an invariant of Algorithm 2.*

PROOF. The proof by induction is omitted for lack of space. □

THEOREM 7.3. *Algorithm 1 is a wait-free, linearizable implementation of a checkpointable object, i.e., an adaptive, RMWable, single-scanner snapshot object. It supports UPDATE, CLICK, and OBSERVE in $O(1)$ time per operation, and has a space complexity of $O(m)$ for an instantiation with m components.*

PROOF. The claims of wait-freedom, space complexity and time complexity are immediate from the pseudo-code of Algorithm 1.

To prove the linearizability of Algorithm 1, we need to show that $P \neq \emptyset$ is an invariant of its meta-configuration tracker, i.e., Algorithm 2. This is immediate from the invariance of $\mathcal{I}_2$ for Algorithm 2, which states that $Q \subseteq P$ and $Q \neq \emptyset$. □

8 ADDITIONAL FEATURES

Below, we discuss two additional features that MemSnap supports. We believe these features can be invaluable to practitioners.

Parallel Observation: Until now, we have only allowed one process to perform observations. Actually, observations can be done in parallel. That is, in a duration of time when there are no clicks, any process can observe the value of any component in the last snapshot. This follows from the insight that if X is not changing, then any observe operation on component i will result in $B[i].seq = X$ by the end of the forwarding on Line 3. One nice consequence of parallel observation is that if the scanner has access to parallelism, then a scan of all the components—i.e., a click followed by an observation of each of the m components—can be done in linear work and constant depth!

Dynamic Addition: Until now, we have discussed the components of snapshot as if they are fixed in number and arranged in

- $Q \subseteq P \ \wedge Q \neq \emptyset$, where the set Q and the possibility p^* that appears in the definition of Q are defined as follows:
  - $Q \triangleq \{p \in P^* \mid \exists S \subseteq \mathit{UnLin}(p^*) \cap \{\pi \in \Pi \mid \widehat{x}_\pi < X\} : \exists \alpha \in \mathit{Perm}(S) : \exists q \in P^* :$
    $\mathit{TransitionsOK}(p^*, \alpha, q) \ \wedge$
    $(p.\text{VAL} = q.\text{VAL}) \ \wedge$
    $(\forall \pi \in \Pi \cup \{\sigma\} : p.\pi = q.\pi) \ \wedge$
    $(\forall j \in [1, m] :$
    $((B[j].seq = X) \ \wedge \ (\forall \pi \in S : i_\pi \neq j) \ \wedge \ (p.\text{SNAP}[j] = B[j].val)) \ \vee$
    $((B[j].seq \neq X) \ \wedge \ (p.\text{SNAP}[j] = p.\text{VAL}[j])) \ \vee$
    $((B[j].seq \neq X) \ \wedge \ (\forall \pi \in S : i_\pi \neq j) \ \wedge \ \exists \mu \in \Pi \cup \{\sigma\} : pc_\mu = 12 \ \wedge \ i_\mu = j \ \wedge \ b_\mu = B[i_\mu] \ \wedge \ p.\text{SNAP}[j] = a_\mu)$
    $)$
    $\}$
  - The possibility p^* that appears in the above definition of Q is defined as follows:
    - $\forall j \in [1, m] : p^*.\text{VAL}[j] = A[j]$
    - $\forall j \in [1, m] : p^*.\text{SNAP}[j] = B[j].val$
    - For all $\pi \in \Pi \cup \{\sigma\}$:

$$p^*.\pi = \begin{cases} \text{ack} & \text{if } pc_\pi = 2 \\ r_\pi & \text{if } pc_\pi \in \{5, 8\} \\ \perp & \text{otherwise} \end{cases}$$

Figure 2: Statement $\mathcal{I}_2$: the *Possibilities Invariant* that captures why the algorithm is linearizable

an array. Actually, the components need not be arranged in any particular way. For instance, rather than being represented by $A[i]$ and $B[i]$, a component can be represented as a struct c with two fields: $c.a$ and $c.b$. With such a representation, MemSnap supports the dynamic addition of new components. To add a new component c' with value v, we simply create a new struct c', and set $c'.a = v$ and $c'.b = (v, 0)$. To see why this works, observe that this would be the state of a component c' if it always existed, but was never updated or observed.

9 FINAL REMARKS

The opening sentence of Afek et al.'s original JACM paper on snapshots propounds the theoretical difficulty and practical importance of instantaneously checkpointing the memory of a distributed system, stating that "obtaining an instantaneous global picture of a system from partial observations made over a period of time as the system state evolves is a fundamental problem in distributed and concurrent computing." [2] The RMWability feature that we introduce in the checkpointable object interface captures this fundamental problem, as it applies to modern multiprocessors. In particular, an RMWable checkpointable object adds checkpointing as an additional feature, while preserving the rich underlying primitives that modern shared-memories already offer. This powerful interface can enable memory backups, checkpointing of parameter weights during the training of deep learning models, and debugging of multicore programs. Additionally, click-and-observe adaptivity makes the interface very flexible.

Our MemSnap algorithm shows that checkpointability admits a concise and efficient solution. Notably, MemSnap is optimal in both space and time complexity; it supports all the operations—RMWable update, click, and observe—in just constant time, and it requires only linear memory in the number of components to achieve this. Furthermore, the constant factors on all of these bounds are small.

Along with time and space optimality, three additional properties—adaptivity, parallel observation, and dynamic addition—make MemSnap an ideal candidate for use in parallel algorithms and systems. Parallel observation implies that a scanner with access to parallel threads can scan all m components in just $O(m)$ work and $O(1)$ depth. In fact, a partial snapshot of any $k \geq 1$ components can be taken in just $O(k)$ work and $O(1)$ depth. Along with enabling standard partial snapshots where the scanner knows which subset of components it wants to observe ahead of time, adaptivity enables dynamic partial snapshots, where the components that are scanned depend on the results of previous observations. For example, consider the problem of finding a key x in a snapshot of a concurrent binary search tree. Adaptivity allows the scanner to observe the values along a spine of the tree, starting at the root, and dynamically choose whether it wants to terminate, observe the left child, or observe the right child of a node—depending on whether $x = v$, $x < v$, or $x > v$. In particular, this means that the entire finding procedure takes just logarithmic time in the size of tree-snapshot. Dynamic additions permit snapshotting, even in the face of memory allocations, e.g., if new nodes get added to the tree.

MemSnap also serves as a great case study for linearizability verification techniques. While the algorithm is concise, its behavior is complex. Reasoning about its correctness is made particularly challenging, since it exhibits far-future linearizability. Consequently, proof techniques that rely on identifying linearization points as they occur—such as the celebrated forward simulation technique—and even proof techniques that rely on identifying an operation's linearization point by its response cannot prove MemSnap's correctness. In this paper, we used Jayanti, Jayanti, Yavuz, and Hernandez's meta-configuration tracking technique [21], which is a complete forward reasoning technique—to prove correctness. It would be interesting to see whether other proof techniques—such as those that use backward reasoning, prophecies, or an aspect-oriented style—can also prove the correctness of this algorithm. Furthermore, proving MemSnap is a good challenge problem for existing and upcoming linearizability proof techniques.

REFERENCES

[1] 2002. AMD 64-Bit Technology AMD x86-64 Architecture Programmer's Manual Volume 3: General-Purpose and System Instructions.

[2] Yehuda Afek, Hagit Attiya, Danny Dolev, Eli Gafni, Michael Merritt, and Nir Shavit. 1993. Atomic Snapshots of Shared Memory. *J. ACM* 40, 4 (Sept. 1993), 873–890. https://doi.org/10.1145/153724.153741

[3] James H Anderson. 1993. Composite registers. *Distributed Computing* 6, 3 (1993), 141–154.

[4] James H Anderson. 1994. Multi-writer composite registers. *Distributed Computing* 7, 4 (1994), 175–195.

[5] James Aspnes and Maurice Herlihy. 1990. Wait-free data structures in the asynchronous PRAM model. In *Proceedings of the second annual ACM symposium on Parallel algorithms and architectures*. ACM, 340–349.

[6] Hagit Attiya, Maurice Herlihy, and Ophir Rachman. 1992. Efficient atomic snapshots using lattice agreement. *Distributed Algorithms* (1992), 35–53.

[7] Hagit Attiya and Ophir Rachman. 1993. Atomic snapshots in O (n log n) operations. In *Proceedings of the twelfth annual ACM symposium on Principles of distributed computing*. ACM, 29–40.

[8] Benyamin Bashari and Philipp Woelfel. 2021. An Efficient Adaptive Partial Snapshot Implementation. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing* (Virtual Event, Italy) *(PODC'21)*. Association for Computing Machinery, New York, NY, USA, 545–555. https://doi.org/10.1145/3465084.3467939

[9] T Chandra and C Dwork. 1993. Using consensus to solve atomic snapshots. *Submitted for Publication* (1993).

[10] Menglei Chen, Yu Hua, Rong Bai, and Jianming Huang. 2023. A Cost-Efficient Failure-Tolerant Scheme for Distributed DNN Training. In *2023 IEEE 41st International Conference on Computer Design (ICCD)*. 150–157. https://doi.org/10.1109/ICCD58817.2023.00031

[11] Khanh Do Ba. 2006. Wait-Free and Obstruction-Free Snapshot. https://digitalcommons.dartmouth.edu/senior_theses/49/

[12] Cynthia Dwork, Maurice Herlihy, Serge A Plotkin, and Orli Waarts. 1992. Time-lapse snapshots. In *Theory of Computing and Systems*. Springer, 154–170.

[13] S Haldar and K Vidyasankar. 1992. Elegant constructions of atomic snapshot variables. *Unpublished manuscript, May* (1992).

[14] Maurice Herlihy. 1991. Wait-free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (Jan. 1991), 124–149.

[15] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (July 1990), 463–492.

[16] Damien Imbs and Michel Raynal. 2011. A Simple Snapshot Algorithm for Multicore Systems. In *2011 5th Latin-American Symposium on Dependable Computing*. 17–24. https://doi.org/10.1109/LADC.2011.11

[17] Intel. 2020. Intel 64 and IA-32 Architectures Software Developer Manuals. https://software.intel.com/content/www/us/en/develop/articles/intel-sdm.html

[18] Amos Israeli, Amnon Shaham, and Asaf Shirazi. 1995. Linear-time snapshot implementations in unbalanced systems. *Theory of Computing Systems* 28, 5 (1995), 469–486.

[19] Prasad Jayanti. 2002. f-arrays: Implementation and applications. In *Proceedings of the twenty-first annual symposium on Principles of distributed computing*. ACM, 270–279.

[20] Prasad Jayanti. 2005. An Optimal Multi-writer Snapshot Algorithm. In *Proceedings of the Thirty-seventh Annual ACM Symposium on Theory of Computing* (Baltimore, MD, USA) *(STOC '05)*. ACM, New York, NY, USA, 723–732. https://doi.org/10.1145/1060590.1060697

[21] Prasad Jayanti, Siddhartha Jayanti, Ugur Yavuz, and Lizzie Hernandez. 2024. A Universal, Sound, and Complete Forward Reasoning Technique for Machine-Verified Proofs of Linearizability. *Proc. ACM Program. Lang.* 8, POPL, Article 82 (jan 2024), 29 pages. https://doi.org/10.1145/3632924

[22] Siddhartha Visveswara Jayanti. 2022. *Simple, Fast, Scalable, and Reliable Multiprocessor Algorithms*. Ph. D. Dissertation. Massachusetts Institute of Technology (MIT), Department of Electrical Engineering and Computer Science. Code available at: https://github.com/visveswara/machine-certified-linearizability.

[23] Lefteris M Kirousis, Paul Spirakis, and Philippas Tsigas. 1991. Reading many variables in one atomic operation solutions with linear or sublinear complexity. In *International Workshop on Distributed Algorithms*. Springer, 229–241.

[24] Nirmesh Malviya, Ariel Weisberg, Samuel Madden, and Michael Stonebraker. 2014. Rethinking main memory OLTP recovery. In *IEEE 30th International Conference on Data Engineering, Chicago, ICDE 2014, IL, USA, March 31 - April 4, 2014*, Isabel F. Cruz, Elena Ferrari, Yufei Tao, Elisa Bertino, and Goce Trajcevski (Eds.). IEEE Computer Society, 604–615. https://doi.org/10.1109/ICDE.2014.6816685

[25] Charles E. McDowell and David P. Helmbold. 1989. Debugging concurrent programs. *ACM Comput. Surv.* 21, 4 (dec 1989), 593–622. https://doi.org/10.1145/76894.76897

[26] Bogdan Nicolae, Jiali Li, Justin M. Wozniak, George Bosilca, Matthieu Dorier, and Franck Cappello. 2020. DeepFreeze: Towards Scalable Asynchronous Checkpointing of Deep Learning Models. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. 172–181. https://doi.org/10.1109/CCGrid49817.2020.00-76

[27] Joakim Öhman and Aleksandar Nanevski. 2022. Visibility reasoning for concurrent snapshot algorithms. *Proc. ACM Program. Lang.* 6, POPL, Article 33 (jan 2022), 30 pages. https://doi.org/10.1145/3498694

[28] Yaron Riany, Nir Shavit, and Dan Touitou. 1995. Towards a practical snapshot algorithm. In *Theory of Computing and Systems, 1995. Proceedings., Third Israel Symposium on the*. IEEE, 121–129.

[29] Yuanhao Wei, Naama Ben-David, Guy E. Blelloch, Panagiota Fatourou, Eric Ruppert, and Yihan Sun. 2021. Constant-time snapshots with applications to concurrent data structures. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Virtual Event, Republic of Korea) *(PPoPP '21)*. Association for Computing Machinery, New York, NY, USA, 31–46. https://doi.org/10.1145/3437801.3441602

[30] Leqi Zhu and Faith Ellen. 2016. Atomic Snapshots from Small Registers. In *19th International Conference on Principles of Distributed Systems (OPODIS 2015) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 46)*, Emmanuelle Anceaume, Christian Cachin, and Maria Potop-Butucaru (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 17:1–17:16. https://doi.org/10.4230/LIPIcs.OPODIS.2015.17